

Capa de acceso a datos

M5: Desarrollo de aplicaciones web dinámicas en Java

|AE4: Implementar capa de acceso a datos en una aplicación web dinámica utilizando el patrón DAO para resolver un problema determinado

Introducción

En el desarrollo de aplicaciones web dinámicas con tecnología J2EE, la capa de acceso a datos juega un papel esencial en la gestión de la persistencia y el acceso a la base de datos. En esta lección, explicaremos **cómo interactuar con la base de datos utilizando JDBC** (Java Database Connectivity), **una API de Java que nos permite realizar operaciones de base de datos desde nuestro código Java**.

Aprenderemos cómo configurar el Driver JDBC para nuestra base de datos, cómo obtener una conexión a la base de datos utilizando el DataSource, y cómo ejecutar consultas SQL para almacenar y recuperar datos. Además, veremos cómo implementar un objeto singleton para gestionar la conexión a la base de datos de manera eficiente.

El uso de JDBC nos proporciona un mayor control sobre la interacción con la base de datos, lo que puede ser especialmente útil para proyectos más pequeños o donde se requiere un alto rendimiento.

Aprendizaje esperado

Al finalizar la lección podrás:

- Implementar capa de acceso a datos en una aplicación web dinámica utilizando el patrón DAO para resolver un problema determinado.

Accesando a datos en una aplicación web

La capa de acceso a datos es la **responsable de manejar la interacción con la base de datos, abstraer la complejidad de las operaciones de base de datos y permitir que la lógica de negocio trabaje con objetos Java en lugar de consultas SQL directas**. Esto mejora la mantenibilidad, la flexibilidad y el rendimiento de la aplicación al mantener una separación clara entre las diferentes capas del sistema.

Sus principales características y roles incluyen:

- **Gestión de la Persistencia:** La capa de acceso a datos se encarga de gestionar la persistencia de los datos de la aplicación en una base de datos. Esto implica la capacidad de almacenar, recuperar y actualizar la información de manera confiable y eficiente.
- **Abstracción del Modelo de Datos:** En lugar de interactuar directamente con tablas y consultas SQL, los componentes de la aplicación trabajan con objetos de dominio o entidades que representan los datos en la base de datos.
- **Data Access Objects (DAO):** La capa de acceso a datos a menudo utiliza el patrón de diseño DAO para encapsular los detalles específicos del acceso a la base de datos. El DAO proporciona una interfaz para realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) en los objetos de entidad. Esto permite un desacoplamiento entre la lógica de negocio y el acceso a datos, lo que facilita la modificación y la reutilización del código.
- **Optimización de Consultas:** La capa de acceso a datos es responsable de optimizar las consultas y las operaciones de base de datos para mejorar el rendimiento de la aplicación. Esto puede incluir el uso de índices, el ajuste de consultas y la utilización de técnicas de caché para reducir el tiempo de acceso a los datos.

Conexión a una Base de Datos mediante la biblioteca JDBC

Java™ Database Connectivity (JDBC) es la **especificación JavaSoft de una interfaz de programación de aplicaciones (API) estándar que permite que los programas Java accedan a sistemas de gestión de bases de datos**. La API

JDBC consiste en un conjunto de interfaces y clases escritas en el lenguaje de programación Java.

Con estas interfaces y clases estándar, los programadores pueden escribir aplicaciones que conecten con bases de datos, envíen consultas escritas en el lenguaje de consulta estructurada (SQL) y procesen los resultados.

Cada propietario de base de datos implementa un driver JDBC que podemos utilizar en nuestras aplicaciones Java. Normalmente habrá un driver por versión y tipo de base de datos, puesto que no siempre se cumple la compatibilidad hacia versiones anteriores.

El driver JDBC es el más básico de que dispone Java para acceder a la base de datos, por lo tanto utilizarlo implica:

- Manejar manualmente las conexiones, nos estamos arriesgando a no cerrar correctamente las conexiones a la base de datos, y un sin fin de problemas derivados. En el caso de aplicaciones web, el servidor de aplicaciones puede que implemente un pool de conexiones automático. Por ejemplo en el caso de Apache Tomcat 6, colocando el driver JDBC en el directorio **lib** del propio Tomcat, no en nuestra aplicación, y configurando el fichero de contexto (context.xml) de nuestra aplicación (o también directamente la configuración global del servidor Tomcat – server.xml) podremos utilizar su pool de conexiones por defecto.
- Cada vez que escribamos una query SQL vamos a tener que escribirla completamente, puesto que cualquier sentencia Insert, Update o Select requerirá escribir un gran número de campos.
- Manejar manualmente las relaciones, teniendo en cuenta el orden de inserción o modificación de tablas y columnas. Esto puede no parecer un gran problema, pero se complica exponencialmente en modelos de datos complejos.
- El SQL lo escribiremos para la versión y tipo de nuestra base de datos, por lo tanto, este código estará completamente acoplado, sin poder ser utilizado sin ser rehecho con otra base de datos.

Componentes de la API JDBC

Driver: Es el enlace que maneja toda la comunicación con la base de datos. Normalmente, una vez que se carga el controlador, el desarrollador no necesita llamarlo explícitamente.

Connection: Es una interfaz con todos los métodos para contactar una base de datos. El objeto de conexión representa el contexto de comunicación, es decir, toda la comunicación con la base de datos es sólo a través del objeto de Connection.

Statement: Encapsula una instrucción SQL que se pasa a la base de datos para ser analizada, compilada, planificada y ejecutada.

ResultSet: Los ResultSet representan un conjunto de filas recuperadas debido a la ejecución de una consulta.

Implementando un objeto Singleton para la conexión a base de datos

Singleton es un **patrón de diseño creacional** que nos permite asegurarnos de que una clase tenga una única instancia, a la vez que proporciona un punto de acceso global a dicha instancia.

Implementar un objeto singleton para la conexión a la base de datos es una **práctica común para asegurar que solo exista una instancia de la conexión y reutilizarla en toda la aplicación**. Esto puede mejorar el rendimiento y la eficiencia, evitando la creación excesiva de conexiones.

Para comunicarnos con una base de datos utilizando JDBC, se debe en primer lugar establecer una conexión con la base de datos a través del driver JDBC apropiado. El API JDBC especifica la conexión en la interfaz java.sql.Connection.

La clase DriverManager permite obtener objetos Connection. Para conectarse es necesario proporcionar:

- URL de conexión, que incluye:
 - Nombre del host donde está la base de datos.
 - Nombre de la base de datos a usar.
- Nombre del usuario en la base de datos.
- Contraseña del usuario en la base de datos.

El siguiente código muestra un ejemplo de conexión y obtención de datos en JDBC a una base de datos MySQL:

```
protected void conectarBase(){
    try {
        //registrar el Driver de MySQL
        Class.forName("com.mysql.jdbc.Driver");

        //URL de la DB
        String urlDataBase = "jdbc:mysql://localhost:3306/"+database+"?useSSL=false";

        //Establecemos la conexion!
        Connection conexion = DriverManager.getConnection(urlDataBase, user, password);

    } catch (Exception e) {
        System.out.println("Error en Conectar Base");
    }
}
```

En este ejemplo, primero se revisa el driver con la sentencia Class.forName.

Después, la clase DriverManager intenta establecer una conexión con la base de datos database utilizando el driver JDBC que proporciona MySQL. Para poder acceder a MySQL es necesario introducir un usuario y un password válidos.

Una vez cargado el driver se debe establecer la conexión con la BD. Para ello se utiliza el siguiente método:

```
Connection conexion = DriverManager.getConnection(url, login, password);
```

La conexión a la BD está encapsulada en un objeto Connection, y para su creación se debe proporcionar la url de la BD y el username y password para acceder a ella. El formato de la url variará según el driver que se utilice.

El objeto Connection representa el contexto de una conexión con la base de datos, es decir:

- Permite obtener objetos Statement para realizar consultas SQL.
- Permite obtener metadatos acerca de la base de datos (nombres de tablas, etc.)
- Permite gestionar transacciones.

Veamos el ejemplo completo respetando el patrón singleton:

```

public class EjemploSingleton {
    //OBJETO CONNECTION ---> Encargado de INICIAR - TENER - MANTENER la conexion
    protected Connection conexion = null;
    //OBJETO RESULTSET ---> Guardar todos los datos que llegan de la DB (las filas de la consulta)
    protected ResultSet resultado = null;
    //OBJETO STATEMENT ---> "Tiene" las consultas... es donde generamos las sentencias a ejecutar!
    protected Statement stmt = null;

    private final String user = "root";
    private final String password= "root";
    private final String database = "nombre_database";

    protected void conectarBase(){
        try {
            //registrar el Driver de MySQL
            Class.forName("com.mysql.jdbc.Driver");

            //URL de la DB
            String urlDataBase = "jdbc:mysql://localhost:3306/"+database+"?useSSL=false";

            //Establecemos la conexion!
            conexion = DriverManager.getConnection(urlDataBase, user, password);
        } catch (Exception e) {
            System.out.println("Error en Conectar Base");
        }
    }
}

```

De manera consecutiva a la creación de una conexión, es necesario desconectar la base. Para esto, vamos a crear un método que verifique si hay alguna transacción abierta la cierre.

```

protected void desconectarBase(){
    try {
        if (resultado!=null) {
            resultado.close();
        }
        if (stmt!=null) {
            stmt.close();
        }
        if (conexion!=null) {
            conexion.close();
        }
    } catch (Exception e) {
        System.out.println("Error en Desconectar Base!");
    }
}

```

Una vez que tengamos la conexión con el objeto Connection, la vamos a usar para crear un objeto Statement, este objeto recibe la consulta para ejecutarla y enviársela a la base de datos. La información que recibimos de la base de datos va a ser capturada por el objeto ResultSet para después poder mostrar la información.

Obtención de datos (objeto Statement)

Una vez realizada la conexión a la BD, se puede utilizar para crear sentencias. Estas sentencias están encapsuladas en la clase Statement, y se pueden crear de la siguiente forma:

```
Statement stmt= conexion.createStatement();
```

- Los objetos Statement permiten realizar consultas SQL en la base de datos.
- Se obtienen a partir de un objeto Connection.

El método para realizar consultas es el método **executeQuery**: envía a la base de datos sentencias SQL para que recuperen datos y devuelven un único objeto ResultSet. Es usado para leer datos (típicamente consultas SELECT).

Una vez obtenido este objeto se puede ejecutar sentencias utilizando su método **executeQuery()** al que se proporciona una cadena con la sentencia SQL que se quiere ejecutar:

```
stmt.executeQuery(sentenciaSQL);
```

Estas sentencias pueden utilizarse para consultas a la base de datos.

Las sentencias SQL se van a escribir entre comillas dobles ya que el objeto Statement recibe String, como dato para las sentencias.

```
String sentenciaSQL = "SELECT nombre, apellido FROM alumnos";
```

Como podemos ver las sentencias van a tener la misma sintaxis que en MySQL, la única diferencia se va a presentar a la hora de trabajar con datos de tipo String y de tipo Date. Como estos datos suelen ir en comillas dobles en Java y en SQL, y nuestra sentencia ya está entre comillas dobles, deberemos poner los datos entre comillas simples para diferenciarlos.

String:

```
"SELECT nombre, apellido FROM Alumnos WHERE nombre = 'Nadin';"
```

Date:

```
"SELECT nombre FROM Alumnos WHERE fechaNacimiento = '07-08-1991';"
```


Obtención de datos (objeto ResultSet)

Para obtener datos almacenados en la BD se utiliza una consulta SQL (query). La consulta se puede ejecutar utilizando el objeto Statement, con el método `executeQuery()` al que se le pasa una cadena con la consulta SQL. **Los datos resultantes se devuelven como un objeto ResultSet.**

```
ResultSet result = stmt.executeQuery(sentenciaSQL);
```

La consulta SQL devolverá una tabla que tendrá una serie de campos y un conjunto de registros, cada uno de los cuales consistirá en una tupla de valores correspondientes a los campos de la tabla.

El objeto ResultSet proporciona el acceso a los datos de estas filas mediante un conjunto de métodos **get** que permiten el acceso a las diferentes columnas de las filas.

El método `ResultSet.next` se usa para moverse a la siguiente fila del ResultSet, convirtiendo a ésta en la fila actual.

El formato general de un ResultSet es una tabla con cabeceras de columna y los valores correspondientes devueltos por la “query”. Por ejemplo, si la “query” es `SELECT a, b, c FROM Table1`, el resultado tendrá una forma semejante a:

a	b	c
12345	Chile	10,5
31245	Brasil	22,7
47899	Perú	56,7

El siguiente fragmento de código es un ejemplo de la ejecución de una sentencia SQL que devolverá una colección de filas, con la columna 1 como un int, la columna 2 como una String y la columna 3 como un valor real:

```
Statement stmt = connection.createStatement();
ResultSet r = stmt.executeQuery("SELECT a, b, c FROM Table1");
while (r.next()) {
    int i = r.getInt("a");
    String s = r.getString("b");
    double d = r.getDouble("c");
    // imprimimos los valores de la fila actual
    System.out.println("Fila = " + i + " " + s + " " + d);
}
```

Filas ResultSet

Un ResultSet mantiene un cursor que apunta a la fila actual de datos. El cursor se mueve una fila hacia abajo cada vez que se llama al método next. Inicialmente se sitúa antes de la primera fila, por lo que hay que llamar al método next para situarlo en la primera fila convirtiéndola en la fila actual.

Las filas de ResultSet se recuperan en secuencia desde la fila más alta a la más baja.

Columnas ResultSet

Los métodos **getX** suministran los medios para recuperar los valores de las columnas de la fila actual. Dentro de cada fila, los valores de las columnas pueden recuperarse en cualquier orden, pero para asegurar la máxima portabilidad, deberían extraerse las columnas de izquierda a derecha y leer los valores de las columnas una única vez.

Puede usarse, o bien el nombre de la columna o el número de columna, para referirse a esta. Por ejemplo: si la columna segunda de un objeto ResultSet rs se denomina "nombre" y almacena valores de cadena, cualquiera de los dos ejemplos siguientes nos devolverá el valor almacenado en la columna.

```
String s = rs.getString("nombre");
```

```
String s = rs.getString(2);
```

Nótese que las columnas se enumeran de izquierda a derecha comenzando con la columna 1.

Además, los nombres usados como input en los métodos getX son insensibles a las mayúsculas.

Algunos de los datos que podemos traer con el método get es:

getInt()	Sirve para obtener un numero entero de la base de datos
getLong()	Sirve para obtener un número long de la base de datos

Modificación de datos

El objeto Statement también posee el método **executeUpdate**: recibe como parámetro la cadena de caracteres que contiene la sentencia SQL a ejecutar. Este método únicamente permite realizar sentencias de actualización de la BD: creación de tablas (CREATE), inserción (INSERT) y borrado de datos (DELETE). El método a utilizar es el siguiente:

```
stmt.executeUpdate(sentenciaSql);
```

Vamos a ver a continuación un ejemplo de estas operaciones. Crearemos una tabla ALUMNOS en nuestra base de datos y añadiremos datos a la misma. La sentencia para la creación de la tabla será la siguiente:

```
String st_crea = "CREATE TABLE ALUMNOS (  
    exp INTEGER,  
    nombre VARCHAR(32),  
    PRIMARY KEY (exp)  
)";
```

```
stmt.executeUpdate(st_crea);
```

Una vez creada la tabla podremos insertar datos en ella como se muestra a continuación:

```
String st_inserta = "INSERT INTO ALUMNOS(exp, nombre)  
    VALUES(1285, 'Manu')";
```

```
stmt.executeUpdate(st_inserta);
```

Cuando tengamos datos dentro de la tabla, podremos modificarlos utilizando para ello una sentencia UPDATE:

```
String st_actualiza = "UPDATE FROM ALUMNOS  
    SET nombre = 'Nadu'  
    WHERE exp = 1285";
```

```
stmt.executeUpdate(st_actualiza);
```

Si queremos eliminar un registro de la tabla utilizaremos una sentencia DELETE como se muestra a continuación:

```
String st_borra = "DELETE FROM ALUMNOS
```

```
WHERE exp = 1285";
```

```
stmt.executeUpdate(st_borra);
```

El método **executeUpdate** retorna un entero que nos dice el número de registros a los que ha afectado la operación, en caso de sentencias INSERT, UPDATE y DELETE. La creación de tablas retorna siempre 0.

La Capa de Acceso a Datos (DAL)

¿Qué es una DAL?

La capa de acceso a datos (**Data Access Layer**) es un componente de una arquitectura de software que es responsable de administrar el almacenamiento y la recuperación de datos de una aplicación.

Se encuentra entre la capa de lógica de negocios y el sistema de almacenamiento de datos y proporciona una capa de abstracción que permite que la capa de lógica de negocios interactúe con el sistema de almacenamiento de datos sin conocer su implementación específica.

El DAL está diseñado para ser reutilizable e independiente de la lógica comercial y la implementación del almacenamiento de datos. Se puede implementar utilizando diferentes tecnologías de acceso a datos y proporciona soporte para transacciones y otras funciones de acceso a datos.

El patrón DAO

El patrón DAO (**Data Access Object**) propone separar por completo la lógica de negocio de la lógica para acceder a los datos, de esta forma, el DAO proporcionará los métodos necesarios para insertar, actualizar, borrar y consultar la información; por otra parte, la capa de negocio solo se preocupa por lógica de negocio y utiliza el DAO para interactuar con la capa de acceso a datos.

¿Qué es un Data Transfer Object (DTO)?

Es un patrón de diseño que se utiliza para transferir datos entre diferentes capas de una aplicación. Es muy común utilizar DTOs en aplicaciones que utilizan una arquitectura de tres capas, donde la capa de presentación, la capa de negocio y la capa de acceso a datos están separadas.

El objetivo de los DTOs es reducir la cantidad de datos que se transfieren entre las diferentes capas de la aplicación. Esto se logra mediante la creación de objetos que contienen solo los datos necesarios para una operación específica. Por ejemplo, si una pantalla de la aplicación solo necesita mostrar el nombre y la dirección de un cliente, el DTO correspondiente solo contendrá esos dos campos.

En cuanto a su implementación, un DTO es simplemente una clase que tiene atributos y métodos getter y setter para acceder a esos atributos. Los DTOs no tienen lógica de negocio y su único propósito es transportar datos.

Ejemplo:

```
public class UsuarioDTO implements Serializable {  
  
    private final String nombre;  
  
    private final String apellidos;  
  
    // constructor y otros métodos  
  
}
```

Por norma general los DTO son clases con atributos con la palabra clave **final**, es decir son clases inmutables. Un DTO no debería de ser capaz de cambiar los datos que almacena ya que sirve para transmitir datos específicos en nuestro proyecto y si los cambiamos de forma errónea podríamos alterar el comportamiento de nuestra aplicación.

Además suelen implementar la interfaz **Serializable** ya que pueden ser utilizados para convertirlos en bytes y enviar u obtener la información de alguna fuente.

Implementando un DAO con métodos CRUD

Veamos un ejemplo de clases DAO. Esto lo vamos a lograr a través de cuatro clases:

1. **Entidad:** va a ser la clase que va a representar a la tabla que queremos trabajar de la base de datos. Va tener como atributos las columnas de la tabla de la base de datos. En este ejemplo la entidad será la clase Usuario.

```
public class Usuario {
    private String nombre;
    private String correoElectronico;
    private Integer edad;
}
```

2. **DAO:** representa una capa de acceso a datos que oculta la fuente y los detalles técnicos para recuperar los datos. Esta clase va a ser la encargada de comunicarse con la base de datos, de conectarse con la base de datos, enviar las consultas y recuperar la información de la base de datos.

Es la clase donde tenemos los objetos de conexión, sentencia y resultado.

```
public class DAO {
    //OBJETO CONNECTION ---> Encargado de INICIAR - TENER - MANTENER la conexion
    protected Connection conexion = null;
    //OBJETO RESULTSET ---> Guardar todos los datos que llegan de la DB (las filas de la consulta)
    protected ResultSet resultado = null;
    //OBJETO STATEMENT ---> "Tiene" las consultas... es donde generamos las sentencias a ejecutar!
    protected Statement stmt = null;

    private final String user = "root";
    private final String password = "root";
    private final String database = "nombre_database";

    protected void conectarBase(){}

    protected void consultarBase(String sql) {}

    protected void desconectarBase(){}
}
```

3. **EntidadDAO:** esta clase va a extender de la clase DAO y se va encargar de generar las sentencias SQL para enviar a la clase DAO, como un insert, select, etc. Y si necesitamos ejecutar un select, sería también, la encargada de recibir la información que recupera la clase DAO de la base de datos sobre una entidad, para después enviarla al servicio, que será la

encargada de imprimir dicha información. Este es un objeto plano que implementa el patrón Data Transfer Object (DTO), el cual sirve para

```
public class UsuarioDAO extends DAO {  
    public List<Usuario> listarUsuario() throws Exception{  
        try {  
            //armado de la sentencia SQL  
            String sql = "SELECT nombre,correoElectronico,edad "  
                + "FROM Usuario; ";  
  
            //consultamos la DB haciendo uso del método del DAO  
            consultarBase(sql);  
  
            //Recorro el resultado de la consulta  
            Usuario usuario = null;  
            List<Usuario> usuarios = new ArrayList<>();  
  
            while(resultado.next()){  
                usuario = new Usuario();  
  
                usuario.setNombre(resultado.getString(2));  
                usuario.setCorreoElectronico(resultado.getString(3));  
                usuario.setEdad(resultado.getInt(4));  
  
                //lo agrego a la list!  
                usuarios.add(usuario);  
            }  
  
            return usuarios;  
        }  
    }  
}
```

transmitir la información entre el DAO y el Business Service.

4. **Servicio o Business Service:** va a tener toda la lógica de negocio del proyecto, usualmente se genera una para cada entidad. Es la que se encarga de obtener datos desde la base de datos y enviarla al cliente, o a su vez recibir la clase desde el cliente y enviar los datos al servidor, por lo general tiene todos los métodos CRUD (create, read, update y delete).

```

public class UsuarioService {
    //llamado a UsuarioDAO
    private UsuarioDAO dao;
    // constructor para inicializarlo

    public UsuarioService() {
        this.dao = new UsuarioDAO();
    }

    public List<Usuario> listarUsuario() throws Exception{
        try {

            List<Usuario> usuarios = dao.listarUsuario();

            return usuarios;

        } catch (Exception e) {
            throw new Exception("Error en listar usuario Servicio");
        }
    }
}

```

De ésta manera estaríamos implementando un DAO utilizando las 4 clases que se comunican entre sí.

Optimización de sentencias: PreparedStatement

Cuando se quiere invocar una determinada sentencia repetidas veces, puede ser conveniente dejar esa sentencia preparada para que pueda ser ejecutada de forma más eficiente. Para hacer esto se utiliza la interfaz PreparedStatement, que podrá obtenerse a partir de la conexión a la BD de la siguiente forma:

```
PreparedStatement ps = con.prepareStatement("SELECT * FROM nombreTabla
WHERE campo2 > 1200 AND campo2 < 1300");
```

A este objeto, a diferencia del objeto Statement visto anteriormente, se le proporciona la sentencia SQL en el momento de su creación, por lo que estará preparado y optimizado para la ejecución de dicha sentencia posteriormente.

Sin embargo, lo más común es que se necesite hacer variaciones sobre la sentencia, ya que normalmente no será necesario ejecutar repetidas veces la misma sentencia exacta, sino variaciones de ella. Por ello, este objeto nos permite parametrizar la sentencia.

Para poder parametrizar una sentencia se deben establecer las posiciones de los parámetros con el carácter '?' dentro de la cadena de la sentencia, tal como se muestra a continuación:


```
PreparedStatement ps = con.prepareStatement("UPDATE FROM nombreTabla  
SET campo1 = 'valor'
```

```
WHERE campo2 > ? AND campo2 < ?");
```

En este caso se tienen dos parámetros, que representan un rango de valores en el cual se quiere actualizar. Cuando se ejecute esta sentencia, el campo1 de la tabla nombreTabla se establecerá a valor1 desde el límite inferior hasta límite superior indicado en el campo2.

Para dar valor a estos parámetros se utiliza el método setXXX() donde XXX será el tipo de los datos que asignamos al parámetro, indicando el número del parámetro (que empieza desde 1) y el valor que le queremos dar. Por ejemplo, para asignar valores enteros a los parámetros se debe hacer:

```
ps.setInt(1,1200);
```

```
ps.setInt(2,1300);
```

Una vez asignados los parámetros, se puede ejecutar la sentencia llamando al método

executeUpdate() del objeto PreparedStatement:

```
int n = ps.executeUpdate();
```

Igual que en el caso de los objetos Statement, se puede utilizar cualquier otro de los métodos para la ejecución de sentencias, executeQuery() o execute(), según el tipo de sentencia que se vaya a ejecutar.

Cierre

En esta lección, has adquirido conocimientos clave sobre la capa de acceso a datos en aplicaciones web dinámicas J2EE utilizando JDBC. Has aprendido a configurar el Driver JDBC, obtener conexiones a la base de datos mediante el DataSource, y ejecutar consultas SQL para gestionar la persistencia de datos.

Además, has explorado cómo implementar un objeto singleton para optimizar el manejo de la conexión y mejorar el rendimiento de tu aplicación.

Este conjunto de habilidades te permitirá interactuar directamente con la base de datos, brindándote un mayor control y flexibilidad en el acceso a datos. Si bien JDBC ofrece un enfoque más manual y detallado, es una opción sólida para proyectos que requieren un alto rendimiento o donde se busca una mayor personalización en las operaciones de base de datos.

¡Nos vemos de nuevo, más adelante! 🙌

Referencias

- IBM Corporation. (s.f.). *What is JDBC* [Documentación IBM Informix, versión 12.10.0]. Recuperado el 28 de julio de 2025, de

<https://www.ibm.com/docs/es/informix-servers/12.10.0?topic=started-what-is-jdbc>

- GeeksforGeeks. (2025, 23 de julio). Data-Access Layer. Recuperado el 28 de julio de 2025, de <https://www.geeksforgeeks.org/data-access-layer/>

¡Muchas gracias!

Nos vemos en la próxima lección

